Hospital Queue & Bed Management System — 10-Day Build Plan (Production-Shaped)

Team of 3 · Day 1 = Sat 24 Oct 2026 · Day 10 = Mon 2 Nov 2026

1. Roles

Each person owns one domain end to end (database, API, UI, tests) and gets an equal share of HTML, CSS, React, JS, Node and SQL.

Person	Domain
A	Beds & Admissions
B	OPD Queue & Wait-time Analytics
C	Platform, Patients, Auth & City-wide View

2. Split by technology

Tech	Person A	Person B	Person C
SQL	`ward`, `bed`, `bed_assignment`, `admission`, `admission_request` migrations; partial unique indexes (one active assignment per bed and per admission); composite FK so a bed and its admission can't cross hospitals; bed-status transition trigger; `reserved_until`; ward-occupancy view	`opd_queue`, `queue_entry`, `queue_counter`, `doctor_session`, `consultation_log` migrations; atomic token counter; call-next query with priority aging; arrival-rate, service-time and live-doctor-count views	`app_user`, `user_hospital_role`, `audit_log` (append-only), `idempotency_key`, `patient` (UUID, UHID, date of birth, merge pointer), `city`, `hospital`, `department`, `referral`; RLS policies; restricted DB roles (`api_user`, `city_reader`); `seed.sql`

Tech	Person A	Person B	Person C
Node / Express	Bed list/status routes; admit, discharge and transfer with `SKIP LOCKED`; waitlist; housekeeping; reservation-expiry sweeper; bed/ward admin	Queue join, call-next, compare-and-swap status transitions, no-show, "symptoms worse" escalation, queue stats, department and doctor-session admin	Express skeleton, request-id + PHI-free logging, auth (argon2id, lockout, rate limit), RBAC + hospital-scoping middleware, audit helper, idempotency middleware, patient routes (duplicate check, merge), city aggregation, referral routes, FastAPI client with fallback
JS (non-React)	`withTransaction()` helper, bed state-machine helper shared with the UI, concurrency test script	API client, `usePolling` hook with stale-data indicator, wait-range formatting, queue sort/aging utils, k6 load script	JWT/auth helpers, role-guard logic, `Idempotency-Key` generator, zod validation schemas, seed runner, city data formatters
React	Ward bed map, admit/discharge/transfer screens, waitlist view, housekeeping view, reservation countdown	Staff queue board, doctor console, patient token screen with "symptoms worse" button, queue stats widget	App shell + router + route guards, login, user and role admin, patient registration with duplicate warning, city dashboard, referral UI, audit-log viewer
HTML + CSS	Design tokens, status badges (never colour alone), responsive grid, mobile bed map	Queue board styling, waiting-room large-screen display (token numbers only, no names)	Navbar/layout, form styling, consent notice page, dashboard cards/tables, login page, final responsiveness pass

Shared reusable pieces: A owns `Badge` / `Table` / `Modal` components, B owns the API client and polling hook, C owns Express middleware and the app shell.

Python / FastAPI (outside the six core techs), split three ways: B writes the Erlang-C function and the FastAPI `POST /estimate` service; A writes `pytest` cases checked against known Erlang-C values and edge cases (ρ ⩾ 1, 0 doctors); C writes the Node →

FastAPI client with a fallback estimate if the Python service is down.

3. Production-readiness requirements

These turn the prototype into something a hospital's IT team could review: every row has an owner and a day, and the work is already inside the timeline below.

Requirement	What gets built	Owner	Day
Identity & access	`app_user`, per-hospital roles, argon2id, lockout, login rate limit, JWT, role guards	C	2
Real authorization	Every query scoped by hospital; API runs as a non-owner DB role with RLS; city dashboard reads one aggregate view through `city_reader`	C	3–4
Audit trail	Append-only `audit_log` written on view, admit, transfer, discharge, queue and referral actions; admin viewer	C (helper), A and B (call it)	2–7
Patient identity	UUID ids, UHID per hospital, date of birth, duplicate warning, merge pointer	C	1–2
Bed integrity	`bed_assignment` history, composite FK, status-transition trigger, reservation expiry sweeper	A	1–4
Safe retries and races	`Idempotency-Key` on admit, join and referral; atomic token counter; compare-and-swap status updates; `row_version`	C (idempotency), A (beds), B (queue)	3
Emergency and fairness	Defined priority levels, aging so low priority is never starved, audited emergency flag, "symptoms worse" escalation	B	3–7

Requirement	What gets built	Owner	Day
Honest wait estimates	`doctor_session` gives live `c`; FastAPI returns a range behind a shared secret; Node falls back if it is down	B (service), C (client)	4
Privacy	Waiting-room display shows tokens only; no PHI in logs or URLs; masked phone numbers in lists; consent notice at registration	B, C	6–8
Migrations and CI	All SQL through `node-pg-migrate`; GitHub Actions builds a fresh Postgres and runs every test on each PR	C (setup), all	1
Automated tests	Concurrency and bed tests (A), queue and Erlang-C tests (B), authorization and end-to-end tests (C)	A, B, C	3–9
Ops basics	`/health` endpoint, separate dev and prod config, backup-and-restore drill, written paper-fallback downtime procedure	C (drill), A (procedure)	8–9

Documented in the report, not built: full DPDP programme (consent management, data-principal rights, retention schedules), MFA, high availability, ABDM/ABHA integration, appointment scheduling, time-varying wait-time models, clinical validation.

4. Day-by-day timeline

Phase 1 — Foundation (Days 1–2)

Day	Person A	Person B	Person C
1 — Sat 24 Oct	*All together, first 2.5 hours:* create the repo (`db/` , `api/` , `analytics/` , `web/`), Supabase project in an India (Mumbai) region, `node-pg-migrate` , `API.md` (auth header, error format, `Idempotency-Key`), Git rules, GitHub Actions running migrations and tests on every PR. Then: design tokens CSS, shared `Badge` / `Table` / `Modal` ; `ward` , `bed` , `bed_assignment` migrations	`queue_counter` , `opd_queue` , `queue_entry` , `doctor_session` , `consultation_log` migrations; API client + `usePolling` hook; FastAPI skeleton (`/health` , shared-secret header)	Express skeleton, `pg` pool on a non-owner role, request-id, error handler and PHI-free logging; migrations for `city` , `hospital` , `department` , `app_user` , `user_hospital_role` , `audit_log` , `patient` , `idempotency_key` ; Vite + React shell
2 — Sun 25 Oct	Bed routes (list, filter, status change) backed by the DB transition trigger; bed state-machine helper; `admission` and `admission_request` migrations	`POST /queue/join` with atomic token counter (idempotency stubbed until C's middleware lands on Day 3); `GET /queue/:id` ; priority levels defined and written into `API.md`	Auth: register and login (argon2id), lockout, rate limit, JWT; RBAC + hospital-scoping middleware; audit helper; patient routes with UHID and duplicate check; `seed.sql` and seed runner

End of Day 1: migrations apply cleanly on a fresh database in CI and everything is merged to `main` . **End of Day 2:** login works, and a staff user sees only their own hospital's rows.

Phase 2 — Core backend (Days 3–5 morning)

Day	Person A	Person B	Person C
3 — Mon 26 Oct	`admitPatient` / `discharge` / bed transfer with `FOR UPDATE SKIP LOCKED`, `withTransaction()`, writes `bed_assignment` and audit rows; **concurrency test (20 requests vs 5 beds)** runs in CI	`call-next` (priority with aging), compare-and-swap status transitions (`WHERE status = 'WAITING'`), no-show/hold, audited emergency flag; queue tests	Idempotency middleware on admit, join and referral; API switched to the non-owner DB role with RLS on `patient`, `admission`, `queue_entry`; authorization tests (cross-hospital access must fail)
4 — Tue 27 Oct	Waitlist table + auto-assign when a bed frees, housekeeping routes, reservation-expiry sweeper, `pytest` cases for Erlang-C	Erlang-C + FastAPI `/estimate` returning a range from the live `doctor_session` count; rolling-average views; "symptoms worse" escalation endpoint	`city_reader` role + aggregate view + city route; referral routes with expiry; Node → FastAPI client with fallback and shared secret; `audit_log` made append-only
5 — Wed 28 Oct (morning)	**Backend checkpoint, all together:** CI is green, every endpoint has an automated test, API contract frozen	Each person tries to break someone else's authorization: cross-hospital reads, wrong role, replayed request	Fix what the attempts found; tag `backend-v1`

Phase 3 — Frontend (Days 5 afternoon–7)

Day	Person A	Person B	Person C
5 — Wed 28 Oct (afternoon)	Ward bed map + CSS (status never shown by colour alone)	Staff queue board + CSS	App shell, router, route guards, login page

Day	Person A	Person B	Person C
6 — Thu 29 Oct	Admit, discharge and transfer screens; reservation countdown	Doctor calling console; patient token screen and waiting-room display (token numbers only, wait range, "symptoms worse" button)	Patient registration with duplicate warning, consent notice page, user and role admin screen
7 — Fri 30 Oct	Waitlist view, housekeeping view, mobile layout	Queue stats widget, aging indicator, stale-data indicator	City dashboard (aggregates only, with an "as of" time), referral UI, audit-log viewer

If an endpoint isn't ready, build the screen against mock JSON from `API.md` rather than waiting.

Phase 4 — Hardening & freeze (Day 8)

Day	Person A	Person B	Person C
8 — Sat 31 Oct	Bed/ward admin screen; transfer edge cases (double discharge, no free bed); bug fixes	k6 load test on queue join and call-next; wait-time calibration; department and doctor-session admin	Security pass (helmet, CORS allow-list, rate limits, secrets only in env, no PHI in logs); Playwright end-to-end test; backup → restore drill on a clean database

End of Day 8: feature freeze. Full run-through from a fresh database, each flow as a different role: register → join queue → call → complete; admit → transfer → discharge → clean → waitlist auto-assign; city dashboard → referral → accept.

Phase 5 — Testing, docs & demo (Days 9–10)

Day	Person A	Person B	Person C
9 — Sun 1 Nov	Final concurrency and edge-case tests; ER diagram v2 + DFD 2B; written paper-fallback downtime procedure; bed-integrity section of the report	Load-test results and token-uniqueness proof; wait-time model assumptions and limits section; DFD 2A	README (setup, env, migrations, roles); DFD 0/1/2C; threat model, privacy notes and known limitations; report assembly; demo seed data
10 — Mon 2 Nov	Buffer, bug fixes, rehearse bed/admission demo	Buffer, bug fixes, rehearse queue/analytics demo	Buffer, bug fixes, slides, rehearse city/platform demo

5. Scope tiers (cut from the bottom if behind)

- **Must have:** migration-built schema, users/roles/auth with hospital scoping, audit log, bed reservation with locking and `bed_assignment`, queue join and call-next with atomic tokens, Erlang-C range estimate, bed map, queue board, token screen, automated tests in CI.
- **Should have:** waitlist auto-assign, city dashboard on the aggregate-only role, referral flow, admin screens, audit-log viewer, duplicate-patient warning, load test.
- **Could have:** Supabase realtime (use 5-second polling instead), deployment to Vercel/Render, patient merge UI.
- **Documented, not built:** full DPDP programme, MFA, high availability, ABDM/ABHA integration, appointments, time-varying wait-time models, clinical validation.

6. Working rules

1. **Day 1 can't slip.** If migrations don't build a fresh database in CI and the skeleton isn't merged by the end of Day 1, everyone is blocked on Day 2.
2. **Schema changes only through migrations.** No hand-edited tables in the Supabase editor.
3. **Merge to `main` at least once a day.** Every PR is reviewed by one teammate (A reviews B, B reviews C, C reviews A) and CI must be green.
4. **10-minute standup daily:** what I finished, what I'm blocked on.
5. **The Day 5 morning checkpoint is real.** If the backend isn't done, finish it before starting UI and cut a *Should have* feature instead.

6. **Whoever builds an endpoint writes its authorization test** (right role passes, wrong role and other hospital fail).
7. **Feature freeze at the end of Day 8.** Days 9–10 are for testing, docs and the demo only.
8. **Synthetic data only,** and no patient data in logs, URLs or error messages.

7. Definition of done

- 20 simultaneous admit requests against 5 beds produce exactly 5 admissions and 15 rejections or waitlist entries, and repeating one request with the same `Idempotency-Key` creates one admission.
- A staff user at hospital X cannot read or change hospital Y's patients, beds or queues (automated test); the city viewer sees aggregates only.
- Every admit, transfer, discharge, queue call and referral writes an `audit_log` row with the actor.
- Queue tokens are unique per department per day, call-next respects priority, and low-priority patients are eventually called.
- Wait-time estimates are a range from Erlang-C using the live doctor count, with a fallback when the service is down; the waiting-room screen shows token numbers only.
- All flows run end to end in the demo, each as a different role.
- A fresh database builds from migrations in CI, and one backup → restore drill has been run.
- ER diagram v2, DFDs (Level 0, 1, 2A–2C), README, downtime procedure, known-limitations section and report are final.